



OAuth 2.0 - JWT

Created by	 Ankit Jain
Category	

Integrate two Salesforce Orgs Using JWT Flow

1. Open a terminal (macOS and Linux) or command prompt (Windows).
2. If necessary, install OpenSSL on your computer. To check whether OpenSSL is installed on your computer, run the `openssl` command on macOS or Linux or the `openssl` command on Windows.

```
which openssl
```

3. Create a directory for storing the generated files, and change to the directory.

```
mkdir JWT
```

```
cd JWT
```

4. Generate a private key, and store it in a file called `server.key`.

```
openssl genpkey -aes-256-cbc -algorithm RSA -pass pass:SomePassword -out server.pass.key -pkeyopt rsa_keygen_bits:2048
```

```
openssl rsa -passin pass:SomePassword -in server.pass.key -out server.key
```

5. Generate a certificate signing request by using the server.key file. Store the certificate signing request in a file called server.csr. Enter information about your company when prompted.

```
openssl req -new -key server.key -out server.csr
```

6. Generate a self-signed digital certificate from the server.key and server.csr files. Store the certificate in a file called server.crt

```
openssl x509 -req -sha256 -days 365 -in server.csr -signkey server.key -out server.crt
```



Salesforce requires certificates in **Java Keystore (JKS)** format for most core configuration workflows, but many third-party authorities issue certificates in **PKCS#12 (.p12 or .pfx)** format. You can bridge this gap by **using a combination of OpenSSL and the Java Keytool utility to convert, extract, or prepare your files for Salesforce**

Convert server.crt created in step 6 into PKS file

7. Clone the **server.key** file and save it as **server.pem**.
8. Now execute this command : (minimum password length 6 char)

```
openssl pkcs12 -export -in server.crt -inkey server.pem -out keystore.p12
```

9. Now execute the keytool command to create the jks file. It will ask you to create a password and remember it.

```
keytool -importkeystore -srckeystore keystore.p12 -srcstoretype pkcs12 -  
destkeystore servercert.jks -deststoretype JKS
```

10. Now Salesforce doesn't support default alias 1. So change the alias name with this command

```
keytool -keystore servercert.jks -changealias -alias 1 -des  
talias "OrgBCert"
```

11. Create a Self-Signed Certificate

```
Setup  
→ Certificate and Key Management  
→ Create Self-Signed Certificate
```

12. Enable Identity Provider

```
Setup  
→ Identity Provider  
→ Enable Identity Provider
```

13. Import a JKS (Java Keystore)

```
Setup  
→ Certificate and Key Management  
→ Import from Keystore
```

Create a Connected App in Your Org - For JWT token purposes

14. Integration User (Org B)

STEP 3: CREATE (OR REUSE) INTEGRATION USER (in Org B)

IF YOU ALREADY HAVE THE INTEGRATION USER FROM LECTURE 10:

- ☒ Skip this step! Reuse "OrgA Integration" user.
Same user, same permissions, same identity.

IF YOU'RE STARTING FRESH:

NAVIGATION: Setup → Users → New User

First Name:	OrgA
Last Name:	Integration
Email:	your-email+integration@gmail.com
Username:	orga.integration@yourdomain.com (globally unique)
User License:	Salesforce
Profile:	System Administrator (for learning)

→ Save

15. Create Connected App with Certificate (Org B)

CREATE CONNECTED APP (in Org B) – WITH CERTIFICATE

NAVIGATION: Setup → App Manager → New Connected App

BASIC INFORMATION:

Connected App Name:	OrgA JWT Integration
API Name:	OrgA_JWT_Integration
Contact Email:	your-email@company.com

API (ENABLE OAUTH SETTINGS):

☒ Enable OAuth Settings

Callback URL: <https://login.salesforce.com/services/oauth2/callback>
(Required but not used – same as Client Credentials)

☒ Use Digital Signatures ← THIS IS THE KEY DIFFERENCE!
→ Upload the .crt file you downloaded from Org A (Step 2)

Selected OAuth Scopes:

→ Add: "Manage user data via APIs (api)"
→ Add: "Perform requests at any time (refresh_token, offline_access)"

→ Save

16. Pre-authorize the User (Org B)

NAVIGATION: Setup → App Manager → "OrgA JWT Integration"
→ Dropdown (▼) → Manage

OAUTH POLICIES:

Permitted Users: "Admin approved users are pre-authorized"

→ Save

ASSIGN PROFILE/PERMISSION SET:

Scroll to Profiles section → Manage Profiles

→ Add "System Administrator"

(or the Integration User's profile)

→ Save

17. Create External Credential , Named Credential and Permission Set(Org A)

 **SETUP > NAMED CREDENTIALS**
OrgB JWT Auth

Label	Name
OrgB JWT Auth	OrgB_JWT_Auth
Authentication Protocol	
OAuth 2.0	
Authentication Flow Type	Scope
JWT Bearer Flow	
Identity Provider URL	
https://login.salesforce.com/services/oauth2/token	
Signing Certificate	JWT Expiration (Seconds) ⓘ
orgbcert	300
Signing Algorithm	
RS256	

Edit Principal

* Parameter Name

Admin

* Sequence Number

1

* Identity Type

Named Principal

Scope

Principal Access

Name	Entity Type	ID
JWT_PS	PermissionSet	OPSdM00000ZBh0fWAD



SETUP > NAMED CREDENTIALS

OrgB JWT

Label

OrgB JWT

Name

OrgB_JWT

URL

<https://orgfarm-673d72ba36-dev-ed.develop.my.salesforce.com>

Enabled for Callouts



Authentication

External Credential

[OrgB JWT Auth](#)

Client Certificate

Callout Options

Generate Authorization Header 



18. The Apex Code

<https://github.com/techjourneywithankit/Udemy-Integration-TJWA/blob/main/force-app/main/default/classes/OrgBJWTService.cls>